

Exp. No. 6

Develop a lexical Analyzer to test whether a given identifier is valid or not using C.

```
#include <stdio.h>

#include <ctype.h>

#include <string.h>

int main() {

    char a[10];

    int flag = 1, i = 1;

    printf("Enter an identifier: ");

    fgets(a, sizeof(a), stdin);

    // Remove newline character

    a[strcspn(a, "\n")] = '\0';

    if (isalpha(a[0])) {

        while (a[i] != '\0') {

            if (!isdigit(a[i]) && !isalpha(a[i])) {

                flag = 0;

                break;

            }

            i++;

        }

    } else {

        flag = 0;

    }

    if (flag == 1)

        printf("Valid identifier\n");

    else

        printf("Not a valid identifier\n");

    return 0;
```

}

```
1 #include <stdio.h>
2 #include <ctype.h>
3 #include <string.h>
4- int main() {
5     char a[10];
6     int flag = 1, i = 1;
7     printf("Enter an identifier: ");
8     fgets(a, sizeof(a), stdin);
9     // Remove newline character
10    a[strcspn(a, "\n")] = '\0';
11-    if (isalpha(a[0])) {
12-        while (a[i] != '\0') {
13-            if (!isdigit(a[i]) && !isalpha(a[i])) {
14-                flag = 0;
15-                break;
16-            }
17-            i++;
18-        }
19-    } else {
20        flag = 0;
21    }
22    if (flag == 1)
23        printf("Valid identifier\n");
24    else
25        printf("Not a valid identifier\n");
26    return 0;
}
```

Enter an identifier: cd34
Valid identifier

=== Code Execution Successful ===

Exp. No. 7

Write a C program to find FIRST() - predictive parser for the given grammar

```
#include<stdio.h>
```

```
#include<ctype.h>
```

```
void FIRST(char[],char );
```

```
void addToResultSet(char[],char);
```

```
int numOfProductions;
```

```
char productionSet[10][10];
```

```
int main()
```

```
{
```

```
int i;
```

```
char choice;
```

```
char c;
```

```
char result[20];
```

```
printf("How many number of productions ? :");
```

```
scanf(" %d",&numOfProductions);
```

```
for(i=0;i<numOfProductions;i++)//read production string eg: E=E+T
```

```

{
printf("Enter productions Number %d : ",i+1);
scanf(" %s",productionSet[i]);
}
do
{
printf("\n Find the FIRST of :");
scanf(" %c",&c);
FIRST(result,c); //Compute FIRST; Get Answer in 'result' array
printf("\n FIRST(%c)= { ",c);
for(i=0;result[i]!='\0';i++)
printf(" %c ",result[i]); //Display result
printf("}\n");
printf("press 'y' to continue : ");
scanf(" %c",&choice);
}
while(choice=='y'||choice=='Y');
}
/*
*Function FIRST:
*Compute the elements in FIRST(c) and write them
*in Result Array.
*/
void FIRST(char* Result,char c)
{
int i,j,k;
char subResult[20];
int foundEpsilon;
subResult[0]='\0';
Result[0]='\0';
//If X is terminal, FIRST(X) = {X}.

```

```

if(!(isupper(c)))
{
addToResultSet(Result,c);
return ;
}

//If X is non terminal

//Read each production
for(i=0;i<numOfProductions;i++)
{
//Find production with X as LHS
if(productionSet[i][0]==c)
{
//If  $X \rightarrow \epsilon$  is a production, then add  $\epsilon$  to FIRST(X).
if(productionSet[i][2]=='$') addToResultSet(Result,$');
//If X is a non-terminal, and  $X \rightarrow Y_1 Y_2 \dots Y_k$ 
//is a production, then add a to FIRST(X)
//if for some i, a is in FIRST( $Y_i$ ),
//and  $\epsilon$  is in all of FIRST( $Y_1$ ), ..., FIRST( $Y_{i-1}$ ).
Else
{
j=2;
while(productionSet[i][j]!='\0')
{
foundEpsilon=0;
FIRST(subResult,productionSet[i][j]);
for(k=0;subResult[k]!='\0';k++)
addToResultSet(Result,subResult[k]);
for(k=0;subResult[k]!='\0';k++)
if(subResult[k]=='$')
{
foundEpsilon=1;

```

```

break;}

//No ε found, no need to check next element

if(!foundEpsilon)

break;

j++;

}}}}

return ;}

/* addToResultSet adds the computed

*element to result set.

*This code avoids multiple inclusion of elements

*/

void addToResultSet(char Result[],char val){

int k;

for(k=0 ;Result[k]!='\0';k++)

if(Result[k]==val)

return;

Result[k]=val;

Result[k+1]='\0';

}

```

main.c	Output
<pre> 1 #include<stdio.h> 2 #include<ctype.h> 3 void FIRST(char[],char); 4 void addToResultSet(char[],char); 5 int numOfProductions; 6 char productionSet[10][10]; 7 int main() 8 { 9 int i; 10 char choice; 11 char c; 12 char result[20]; 13 printf("How many number of productions ? :"); 14 scanf(" %d",&numOfProductions); 15 for(i=0;i<numOfProductions;i++)//read production string eg: E=E+T 16 { 17 printf("Enter productions Number %d : ",i+1); 18 scanf(" %s",productionSet[i]); 19 } 20 do 21 { 22 printf("\n Find the FIRST of :"); 23 scanf(" %c",&c); 24 FIRST(result,c); //Compute FIRST; Get Answer in 'result' array </pre>	<pre> How many number of productions ? :4 Enter productions Number 1 : S=AaAb Enter productions Number 2 : S=BbBa Enter productions Number 3 : A=\$ Enter productions Number 4 : B=\$ Find the FIRST of :S FIRST(S)= { \$ a b } press 'y' to continue : y Find the FIRST of :A FIRST(A)= { \$ } press 'y' to continue : B === Code Execution Successful === </pre>

Exp. No. 8

Write a C program to find FOLLOW() - predictive parser for the given grammar

```
#include <stdio.h>

#include <ctype.h>

#include <string.h>

int limit, x = 0;

char production[10][10], array[10];

void find_first(char ch);

void find_follow(char ch);

void Array_Manipulation(char ch);

int main()

{

    int count;

    char option, ch;

    printf("Enter Total Number of Productions: ");

    scanf("%d", &limit);

    for(count = 0; count < limit; count++)

    {

        printf("Enter Production %d: ", count + 1);

        scanf("%s", production[count]);

    }

    do

    {

        x = 0;

        printf("\nEnter Non-Terminal to Find FOLLOW: ");

        scanf(" %c", &ch);

        find_follow(ch);

        printf("FOLLOW(%c) = { ", ch);

        for(count = 0; count < x; count++)
```

```

{
    printf("%c ", array[count]);
}
printf("\n");
printf("Continue? (Y/N): ");
scanf(" %c", &option);
} while(option == 'Y' || option == 'y');
return 0;
}

void find_follow(char ch)
{
    int i, j, length;
    // Start symbol gets $
    if(production[0][0] == ch)
    {
        Array_Manipulation('$');
    }
    for(i = 0; i < limit; i++)
    {
        length = strlen(production[i]);
        for(j = 2; j < length; j++) {
            if(production[i][j] == ch)
            {
                // If symbol exists after ch
                if(production[i][j + 1] != '\0')
                {
                    find_first(production[i][j + 1]);
                }
                // If ch is at the end
                if(production[i][j + 1] == '\0' &&
                    ch != production[i][0])

```

```

        {
            find_follow(production[i][0]);
        }
    }
}

}

void find_first(char ch)
{
    int k;
    // If terminal
    if(!isupper(ch))
    {
        Array_Manipulation(ch);
        return;
    }
    for(k = 0; k < limit; k++)
    {
        if(production[k][0] == ch)
        {
            if(production[k][2] == '$')
            {
                find_follow(ch);
            }
            else if(islower(production[k][2]))
            {
                Array_Manipulation(production[k][2]);
            }
        }
        else
        {
            find_first(production[k][2]);
        }
    }
}

```

```

    }
    }
}

void Array_Manipulation(char ch)
{
    int count;

    for(count = 0; count < x; count++)
    {
        if(array[count] == ch)
        {
            return;
        }
    }

    array[x++] = ch;}

```

<pre> 1 #include <stdio.h> 2 #include <ctype.h> 3 #include <string.h> 4 int limit, x = 0; 5 char production[10][10], array[10]; 6 void find_first(char ch); 7 void find_follow(char ch); 8 void Array_Manipulation(char ch); 9 int main() 10 { 11 int count; 12 char option, ch; 13 printf("Enter Total Number of Productions: "); 14 scanf("%d", &limit); 15 for(count = 0; count < limit; count++) 16 { 17 printf("Enter Production %d: ", count + 1); 18 scanf("%s", production[count]); 19 } 20 do 21 { 22 x = 0; 23 printf("\nEnter Non-Terminal to Find FOLLOW: "); 24 scanf(" %c", &ch); 25 find_follow(ch); 26 printf("FOLLOW(%c) = { ", ch); </pre>	<pre> Enter Total Number of Productions: 4 Enter Production 1: S=AaAb Enter Production 2: S=BbBa Enter Production 3: A=\$ Enter Production 4: B=\$ Enter Non-Terminal to Find FOLLOW: S FOLLOW(S) = { \$ } Continue? (Y/N): Y Enter Non-Terminal to Find FOLLOW: A FOLLOW(A) = { a b } Continue? (Y/N): Y Enter Non-Terminal to Find FOLLOW: B FOLLOW(B) = { b a } Continue? (Y/N): b === Code Execution Successful === </pre>
--	--

Exp. No. 9

Implement a C program to eliminate left recursion from a given CFG.

```
#include<stdio.h>

#include<string.h>

#define SIZE 10

int main () {

char non_terminal;

char beta,alpha;

int num;

char production[10][SIZE];

int index=3; /* starting of the string following "->" */

printf("Enter Number of Production : ");

scanf("%d",&num);

printf("Enter the grammar as E->E-A :\n");

for(int i=0;i<num;i++){

scanf("%s",production[i]);

}

for(int i=0;i<num;i++){

printf("\nGRAMMAR : : %s",production[i]);

non_terminal=production[i][0];

if(non_terminal==production[i][index]) {

alpha=production[i][index+1];

printf(" is left recursive.\n");

while(production[i][index]!=0 && production[i][index]!='|')

index++;

if(production[i][index]!=0) {

beta=production[i][index+1];

printf("Grammar without left recursion:\n");
```

```

printf("%c->%c%c\\",non_terminal,beta,non_terminal);

printf("\\n%c\\->%c%c\\'|E\\n",non_terminal,alpha,non_terminal);}

else

printf(" can't be reduced\\n");

}

else

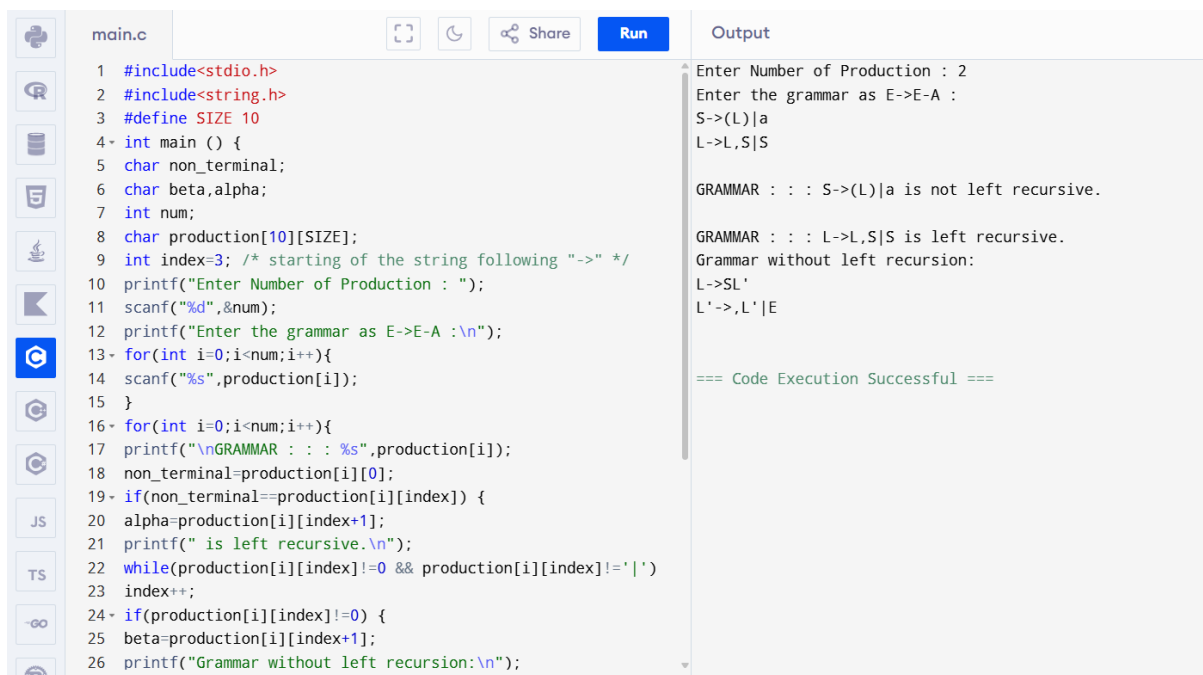
printf(" is not left recursive.\\n");

index=3;

}

}

```



The screenshot shows a C code editor with a file named `main.c`. The code implements a grammar checking algorithm. It prompts the user to enter the number of productions and the grammar rules. It then checks if the grammar is left recursive by comparing the non-terminal at the start of a production with the first non-terminal in its right-hand side. If it is left recursive, it prints a message; otherwise, it prints the grammar without left recursion.

```

1  #include<stdio.h>
2  #include<string.h>
3  #define SIZE 10
4  int main () {
5      char non_terminal;
6      char beta,alpha;
7      int num;
8      char production[10][SIZE];
9      int index=3; /* starting of the string following "->" */
10     printf("Enter Number of Production : ");
11     scanf("%d",&num);
12     printf("Enter the grammar as E->E-A :\n");
13     for(int i=0;i<num;i++){
14         scanf("%s",production[i]);
15     }
16     for(int i=0;i<num;i++){
17         printf("\nGRAMMAR : : : %s",production[i]);
18         non_terminal=production[i][0];
19         if(non_terminal==production[i][index]) {
20             alpha=production[i][index+1];
21             printf(" is left recursive.\\n");
22             while(production[i][index]!=0 && production[i][index]!='|')
23                 index++;
24         }
25         if(production[i][index]!=0) {
26             beta=production[i][index+1];
27             printf("Grammar without left recursion:");
28         }
29     }
30 }

```

The output of the program is as follows:

```

Enter Number of Production : 2
Enter the grammar as E->E-A :
S->(L)|a
L->L,S|S

GRAMMAR : : : S->(L)|a is not left recursive.

GRAMMAR : : : L->L,S|S is left recursive.
Grammar without left recursion:
L->SL'
L'->,L'|E

=== Code Execution Successful ===

```

Exp. No. 10

Implement a C program to eliminate left factoring from a given CFG.

```
#include <stdio.h>

#include <string.h>

int main()
{
    char gram[20], part1[20], part2[20];
    char modifiedGram[20], newGram[20];

    int i, j = 0, k = 0, pos = 0;

    // Input production
    printf("Enter Production: S->");
    fgets(gram, sizeof(gram), stdin);

    // Remove newline character
    gram[strcspn(gram, "\n")] = '\0';

    // Extract part1
    for(i = 0; gram[i] != '|' && gram[i] != '\0'; i++, j++)
        part1[j] = gram[i];
    part1[j] = '\0'

    // Extract part2
    for(j = ++i, i = 0; gram[j] != '\0'; j++, i++)
        part2[i] = gram[j];
    part2[i] = '\0';

    // Find common prefix
    for(i = 0; part1[i] == part2[i] && part1[i] != '\0'; i++)
    {
        modifiedGram[k++] = part1[i];
        pos = i + 1;

    }

    // Create modified production
```

```

modifiedGram[k++] = 'X';

modifiedGram[k] = '\0';

// Create new production

j = 0;

for(i = pos; part1[i] != '\0'; i++, j++)

newGram[j] = part1[i];

newGram[j++] = '|';

for(i = pos; part2[i] != '\0'; i++, j++)

    newGram[j] = part2[i];

newGram[j] = '\0';

// Print result

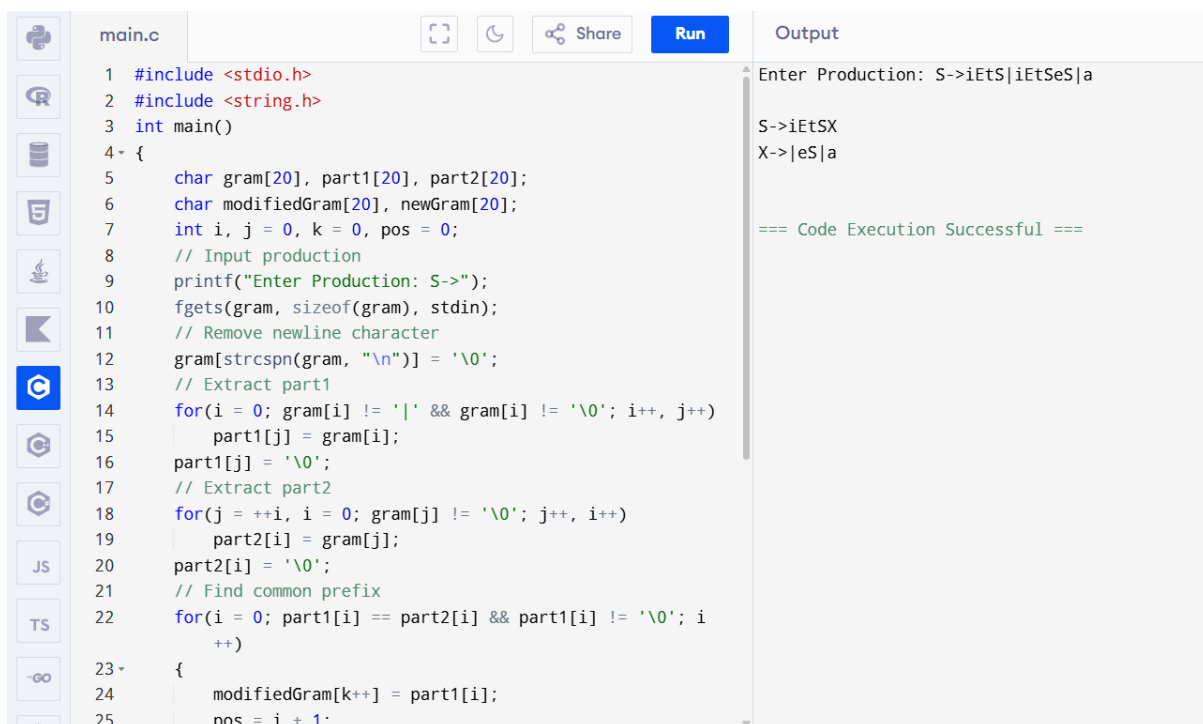
printf("\nS->%s", modifiedGram);

printf("\nX->%s\n", newGram);

return 0;

}

```



The screenshot shows a C code editor with a file named `main.c`. The code implements a function to process a production string. It reads the input "Enter Production: S->iEtS|iEtSeS|a", splits it into two parts "iEtS" and "iEtSeS|a", and then prints the result. The output shows the successful execution of the code.

```

1 #include <stdio.h>
2 #include <string.h>
3 int main()
4 {
5     char gram[20], part1[20], part2[20];
6     char modifiedGram[20], newGram[20];
7     int i, j = 0, k = 0, pos = 0;
8     // Input production
9     printf("Enter Production: S->");
10    fgets(gram, sizeof(gram), stdin);
11    // Remove newline character
12    gram[strcspn(gram, "\n")] = '\0';
13    // Extract part1
14    for(i = 0; gram[i] != '|' && gram[i] != '\0'; i++, j++)
15        part1[j] = gram[i];
16    part1[j] = '\0';
17    // Extract part2
18    for(j = ++i, i = 0; gram[j] != '\0'; j++, i++)
19        part2[i] = gram[j];
20    part2[i] = '\0';
21    // Find common prefix
22    for(i = 0; part1[i] == part2[i] && part1[i] != '\0'; i++)
23    {
24        modifiedGram[k++] = part1[i];
25        pos = i + 1;

```

Output:

```

Enter Production: S->iEtS|iEtSeS|a
S->iEtSX
X->|eS|a

=== Code Execution Successful ===

```